The Midnight Queue: Ranking Suspicious Transactions at Scale

Roll no 24i-2625, Section B | DAA Assignment 2 | seed s = 242625, K = 25 + (242625 mod 226) = 152. Hardware/software: HP EliteBook laptop (7th-gen Intel Core i5) used to run Google Colab; timings measured on the Colab VM (Intel Xeon @ 2.20 GHz, 2 logical CPUs, Python 3.13, numpy 2.1.3, pandas 2.2.3, Linux). Timing: perf_counter, median of 5 runs, DNF limit per run set by MODE. **Definitions:** comparison = one evaluation of "key A before key B"; move = one write of a record into an array slot (swap = 2, self-swap skipped, copies to a buffer count). Records are (key, txn_id); Class never enters a key. **Scope note:** the figures and tables below come from the notebook run in "quick" mode (n = 1,000 and 5,000 for Problem C; n = 10,000 and 50,000 for D; n = 50,000 for E), so no run hit the DNF limit. Absolute times therefore differ from the full-size grid; all derived conclusions are stated at the sizes actually measured.

A. Know your data

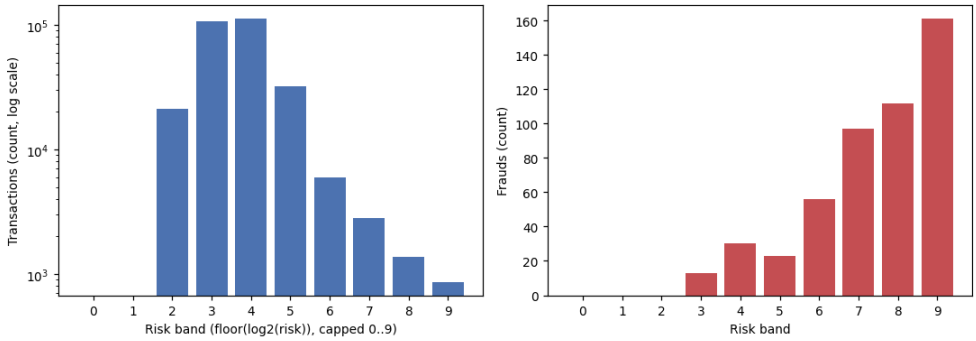


Figure A1. Left: transactions per risk band (log scale). Right: frauds per band (Class used for evaluation only).

Band	2	3	4	5	6	7	8	9
Transactions	21,156	107,410	113,116	32,084	6,006	2,797	1,376	862
Frauds	0	13	30	23	56	97	112	161
Fraud rate %	0	0.012	0.027	0.072	0.93	3.47	8.14	18.68

Only 8 of 10 bands are populated (bands 0-1 are empty: minimum risk is 4.2). Amount has 32,767 distinct values; the 10 most frequent hold 16.29% of rows (Amount = 1.00 alone: 4.81%). 492 frauds (0.173%); 87% of them (426 of 492) sit in bands 6-9, which contain only 3.9% of rows. **Raw-file properties:** the file is exactly C1 (Time non-decreasing). It is not C3/C4/C5: 50.6% of adjacent pairs and 50.1% of random pairs are in triage order (a ranked file gives 100%/0%), and 97.6% of records lie outside the longest ranked subsequence. It is effectively random with respect to the triage key. C6 has only 8 distinct keys, so massive ties.

B. Contenders and correctness

All five sorts and both top-K methods are hand-written (no built-in sort inside). Naive Quick = last-element pivot, Lomuto; Hardened Quick = random pivot + three-way (Dijkstra) partition. Both quicksorts are iterative with an explicit stack (smaller part first), so recursion depth never crashes Python on C3/C4/C6 (naive recursion would reach depth ~n); stack stays O(log n). Heap Sort uses Floyd build. Top-K: (a) full Merge Sort + slice, (b) size-K min-heap (root = worst kept), (c) bounded sorted insertion buffer. **Tests:** 175 edge-case checks pass (empty, single, two records, all-equal, duplicate-heavy, sorted, reverse, Top-K with K < n, = n, > n). Every benchmark run was compared with sorted(): outputs were identical on all conditions and sizes (C6 is not a total order, so stable sorts are compared exactly and unstable ones band by band). Every top-K result equalled sorted()[:K]. Insertion and Merge Sort are stable; Quick and Heap are not.

C. Full ranking tournament

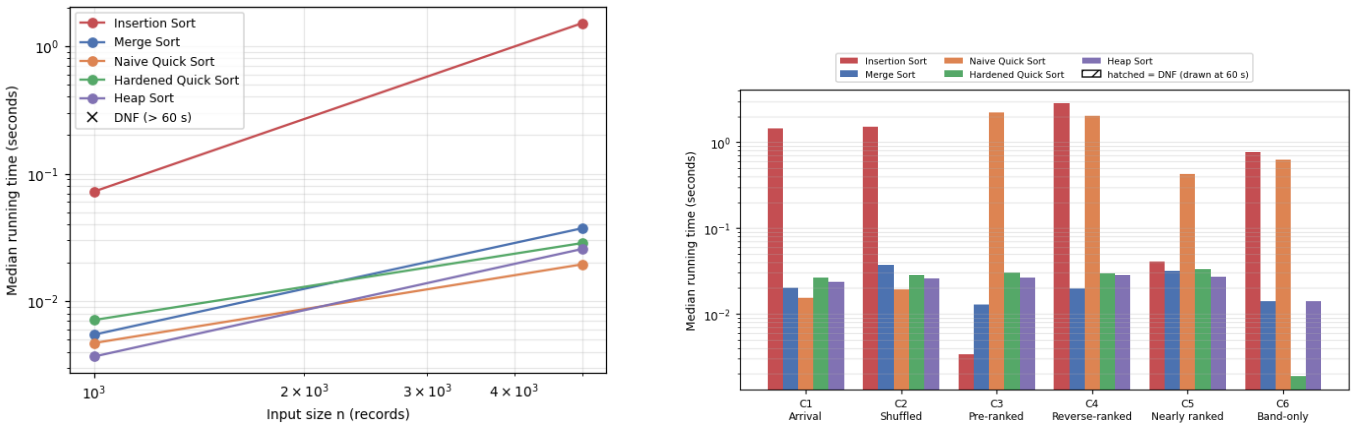


Figure C1 (left): time vs n on C2, log-log, all contenders. Figure C2 (right): time at n = 5,000 (the largest quick-mode size, standing in for 10,000) across the six conditions, log scale.

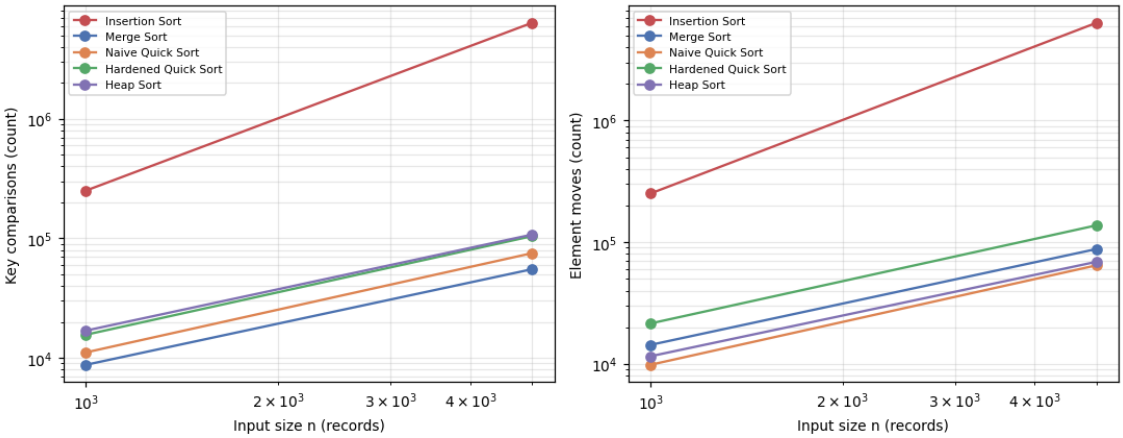


Figure C3. Comparisons (left) and moves (right) vs n on C2, log-log.

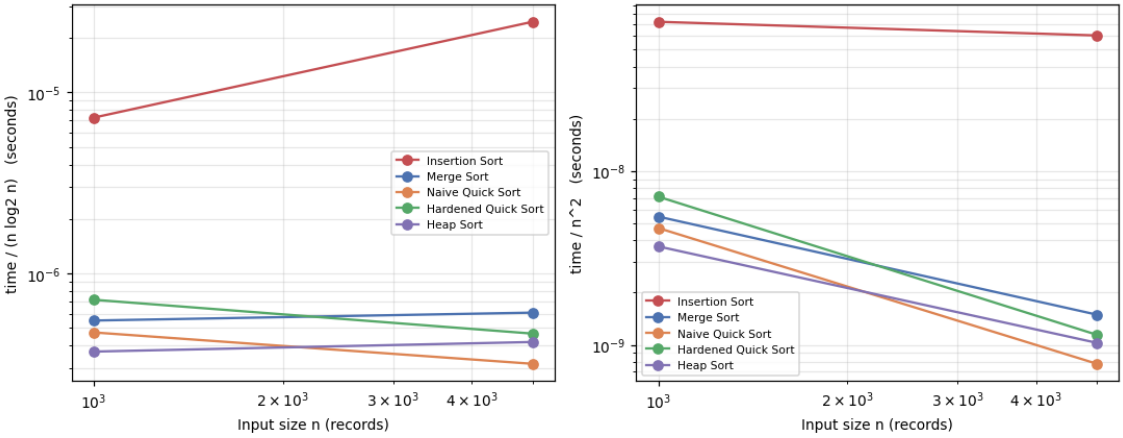


Figure C4. Time / (n log2 n) (left) and time / n^2 (right) vs n on C2.

n=5,000, time s (comparisons)	C1 Arrival	C2 Shuffled	C3 Pre-ranked	C4 Reverse	C5 Nearly	C6 Band-only
Insertion (stable, O(1))	1.466 (6.2M)	1.509 (6.4M)	0.003 (5.0K)	2.853 (12.5M)	0.041 (73.7K)	0.778 (4.3M)
Merge (stable, O(n))	0.020 (55.1K)	0.037 (55.2K)	0.013 (29.8K)	0.020 (32.0K)	0.031 (40.8K)	0.014 (52.4K)
Naive Quick (O(log n) stack)	0.015 (72.1K)	0.019 (75.2K)	2.233 (12.5M)	2.031 (12.5M)	0.431 (2.46M)	0.635 (3.97M)
Hardened Quick (O(log n))	0.027 (99.2K)	0.029 (104K)	0.030 (102K)	0.030 (107K)	0.033 (99.9K)	0.002 (16.4K)
Heap (O(1))	0.024 (108K)	0.026 (108K)	0.027 (112K)	0.028 (103K)	0.027 (112K)	0.014 (87.4K)

Moves (n = 5,000, C2): Insertion 6.37M, Merge 87.8K, Naive Quick 64.5K, Hardened Quick 138K, Heap 69.0K. Extra memory per contender is given by the analysis in M8 (the extra-memory column of the summary). **Reading.** Insertion is quadratic on C1/C2/C4/C6 and linear on C3 (4,999 comparisons, 0 moves). Naive Quick is n log n on random data but exactly n(n-1)/2 = 12,497,500 comparisons on C3 and C4. Hardened Quick and Heap Sort are almost flat across conditions; Merge Sort is the most consistent in absolute terms. The n log n and n^2 normalisations are flat for the matching contenders only over the two small sizes, so growth rates beyond that rest on the counts and on Problem F, not on the timing curve.

Prediction log (appendix, written before the run) vs outcome. Right: Insertion wins C3 and loses C1/C2/C4/C6; Merge wins C4; Naive Quick loses C3 (DNF predicted at n >= 100,000, not reachable in quick mode); Hardened Quick wins C6 by about 7x over Merge. Wrong: (1) I predicted Merge Sort would win C1/C2; Naive Quick won (0.015 s vs 0.020 s, and 0.019 vs 0.037), because it makes fewer element moves per comparison and Python overhead of Merge slicing dominates. (2) I predicted Heap Sort would never win; it won C5 (0.027 s vs Merge 0.031 s, Insertion 0.041 s), a narrow margin that I treat as near-tie, not a real advantage. (3) I predicted Insertion would win C5 only for n up to about 3,000: at n = 1,000 it did (0.002 s vs 0.002 s for Merge), at 5,000 it lost, consistent with n + I versus n log n.

D. The shift queue

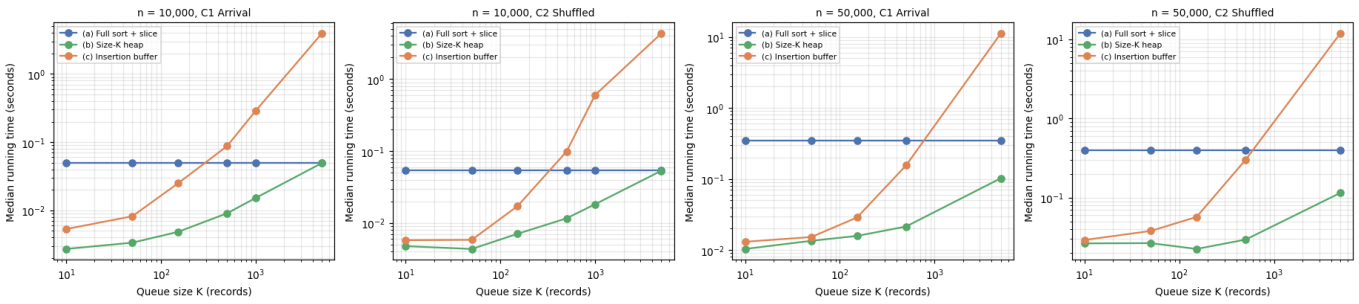


Figure D1. Time to build the top-K queue vs K for each method, log-log, per (n, condition); crosses mark DNF.

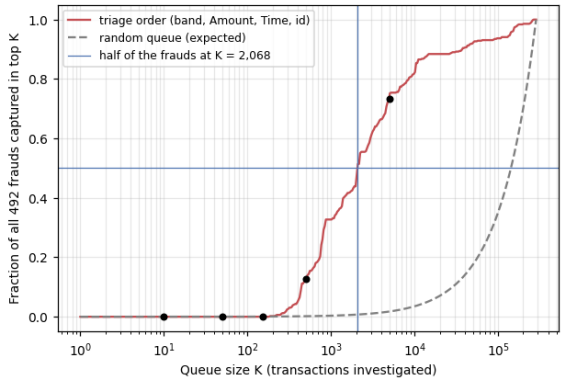


Figure D2. Fraction of all 492 frauds captured in the top K under the triage rule (Class used for evaluation only).

The size-K heap is the fastest method at every K measured except n = 10,000, K = 5,000, where full sort + slice is level in time (0.050 s vs 0.049 s) and ahead in comparisons (120K vs 195K), so the heap stops being best between K = 1,000 and 5,000, i.e. K between n/10 and n/2 at n = 10,000. The insertion buffer was never fastest at K >= 10 (K = 10, n = 10,000: 0.005 s vs heap 0.003 s) and explodes quadratically (K = 5,000: 3.9-4.3 s at n = 10,000, 11.2-11.9 s at n = 50,000). Full sort never wins for K << n (n = 50,000, K = 152: 0.344 s vs heap 0.016 s). **Half the frauds:** the smallest K capturing 246 of 492 frauds is **K = 2,068** (0.73% of rows). Cost at n = 50,000 (C1/C2): full sort 0.340/0.369 s; heap 0.052/0.074 s; insertion buffer 1.72/2.05 s (comparisons 716K / 199K / 7.2M). **Clash:** the shift capacity K = 152 captures **0 frauds**, and K = 500 only 63, even though 87% of frauds are in bands 6-9. The key sorts by band, then Amount: band 9 alone has 862 rows, and its top-152 by Amount are large legitimate payments. The capacity is therefore too small for this ranking rule; I report this and keep the assigned rule rather than altering the sort key.

E. The live feed

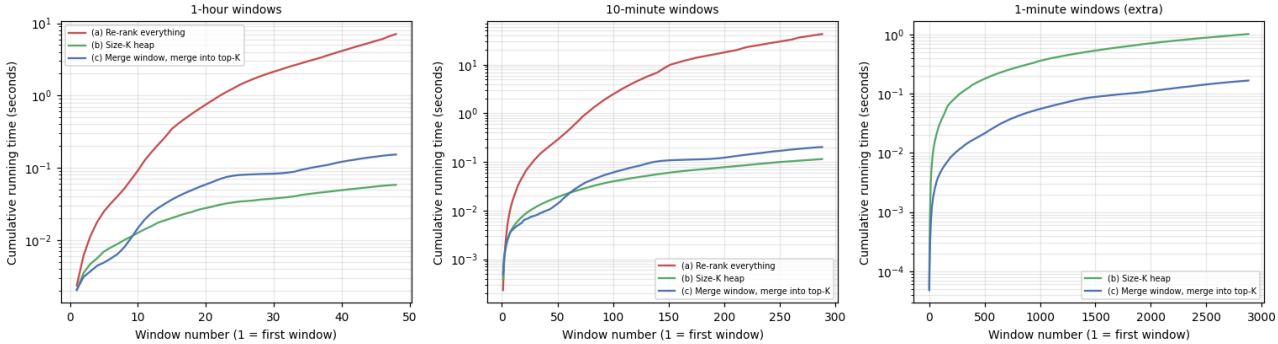


Figure E1. Cumulative time vs window number for the three strategies at 1-hour (48 windows), 10-minute (288) and 1-minute (2,880, extra) windows; n = 50,000, K = 152.

Windows	(a) re-rank all	(b) size-K heap	(c) merge window into top-K	Peak extra memory (a / b / c)
1 hour, w=48	7.09 s (15.1M cmp)	0.058 s (139K)	0.152 s (455K)	1.05 MB / 65 KB / 498 KB
10 min, w=288	42.8 s (88.9M)	0.115 s (527K)	0.203 s (364K)	1.05 MB / 380 KB / 971 KB
1 min, w=2,880	not run	1.020 s (4.71M)	0.167 s (596K)	- / 3.77 MB / 5.94 MB

Published lists matched sorted()[:K] at five checkpoints in every run. Re-ranking everything is 120x (1 h) and 370x (10 min) slower than the heap. **Clash:** at 10-minute windows the model predicts (c) cheaper (364K vs 527K comparisons) but time favours (b) (0.115 s vs 0.203 s), because (c) also pays for a window sort and about 577K moves; by time the crossover is nearer w = 2,880, where (c) wins 6x. The model crossover is w = 145 by comparisons.

F. Mathematical case

M1 Insertion. Running time = Theta(n + I): the inner loop shifts exactly once per inversion, plus one comparison per element. Best case (sorted) I = 0, Theta(n); worst (reversed) I = n(n-1)/2, Theta(n^2); average (random) I ~ n(n-1)/4, Theta(n^2). Measured on C2/C4: comparisons 12,497,500 on C4, n = 5,000. **C5 sample:** I = 3,889 (n = 1,000) and 68,734 (n = 5,000). The model comparisons = I + (n-1) - r and moves = I + t predicts 73,733 and 73,614 at n = 5,000, equal to the measured counts (and 4,888 / 4,715 at n = 1,000).

M2 Merge. T(n) = 2T(n/2) + cn = 4T(n/4) + 2cn = ... = 2^k T(n/2^k) + k*cn; at k = log2 n, T(n) = nT(1) + cn log2 n = Theta(n log n). Master: a = 2, b = 2, f(n) = Theta(n) = Theta(n^(log_b a)), case 2, so Theta(n log n). **M3 Quick.** (a) T(n) = T(n-1) + cn = T(n-2) + c(n-1) + cn = ... = c*n(n+1)/2 = Theta(n^2). (b) Recursion tree: level i has 2^i nodes of size n/2^i, each level costs cn, with log2 n + 1 levels: Theta(n log n). (c) T(n) = T(n/10) + T(9n/10) + n: each level costs at most n; the shortest path has depth log10 n and the longest log_{10/9} n, so n log10 n <= T(n) <= n(log_{10/9} n + 1), hence **Theta(n log n)** (constant ~ 9.5 ln n versus ln n / ln 2). **M4 Heap.** Floyd sifts down every internal node; a node of height h costs <= 2h comparisons and there are <= ceil(n/2^(h+1)) such nodes: sum n*h/2^(h+1)*O(1) = O(n*sum h/2^h) = O(n), since sum h/2^h = 2. Then n-1 extractions at <= 2 log2 n each: total Theta(n log n). **M5 Top-K.** (a) Theta(n log n) regardless of K. (b) one root check per record plus R ~ K ln(n/K) replacements at 2 log2 K each: Theta(n + K log K * log(n/K)), worst case Theta(n log K). (c) R ~ K ln(n/K) insertions at ~ K/2 shifts: Theta(n + K^2 log(n/K)), worst case Theta(nK). Predicted crossovers: buffer worse than heap from K ~ 16 (any n tested); heap worse than full sort from K ~ 18,642 at n = 100,000 (1,864 at n = 10,000, 9,320 at n = 50,000). **Measured:** predicted vs measured comparisons agree within 1-25% (e.g. n = 50,000, K = 152: heap 61.9K/65.1K, buffer 122.7K/123.4K). At n = 10,000 the heap-vs-sort crossover is between K = 1,000 and 5,000 by comparisons against a prediction of 1,864, a fair match. The buffer-vs-heap crossover was measured at K <= 10 (below the grid), while predicted at 16: their comparison counts tie at K = 10, but the heap won on time, so the gap is a Python constant factor, not a mismatch in the asymptotics. The n = 100,000 prediction (K ~ 18,642) was not

measured in quick mode.

n	d	Group sizes m_i	L = sum m_i(m_i-1)/2	Measured	R/n
1,000	8	2,6,13,20,116,410,357,76	157,195	159,716	2.52
5,000	8	14,25,58,109,578,1959,1899,358	3,958,598	3,966,302	1.54

M6 Naive Quick on C6. With the last element as pivot and a strict "<" Lomuto partition, every record equal to the pivot ends on the right, so an all-equal group of size m peels off one element per pass: (m-1)+(m-2)+...+1 = m(m-1)/2 comparisons. Prediction: L = sum m_i(m_i-1)/2 plus R, the cost of passes while groups are still mixed. Measured/L = 1.016 (n = 1,000) and 1.002 (n = 5,000); R is a small linear term (1.5-2.5 comparisons per record). For n = 284,807, L would be on the order of 10^10 comparisons, which explains the DNF predicted for the full file. Three-way partition removes the quadratic term (16,354 comparisons at n = 5,000). **M7 Live feed.** With w equal windows: (a) sum_j (jn/w) log(jn/w) = Theta(n w log n); (b) Theta(n + K ln(n/K) log K + w K log K), the last term being the ranked snapshot each window; (c) Theta(n log(n/w) + wK). (a) is best only at w = 1; (b) is cheapest by comparisons until w ~ 145, after which (c) is cheaper; the crossover moves to larger w for smaller K. Predicted vs measured comparisons agree within 2-36% (10-min: (a) 98.6M vs 88.9M, (c) 353K vs 364K). **M8 Space.** Insertion O(1); Merge O(n); Naive and Hardened Quick O(log n) (explicit stack, smaller part first); Heap O(1); top-K (a) O(n), (b) O(K), (c) O(K); live (a) O(n), (b) O(K), (c) O(W + K), W = window size.

G. Change request addendum

Left for demo.

H. Recommendation for the platform lead

1. Overnight batch (full day, ~285,000 rows): ship Merge Sort (Hardened Quick Sort if memory is tight and stability is unnecessary). Evidence: Merge is Theta(n log n) in every case (M2) and measured at 0.013-0.037 s at n = 5,000 on all six conditions, with the lowest comparison counts and stable ties. Naive Quick Sort was marginally faster on random data but 2.0-2.2 s on pre-ranked and reverse data (n(n-1)/2 comparisons, 150x slower), and nothing guarantees tomorrow's file is random. Assumption: O(n) extra memory is available. Wrong if: memory is capped (use Hardened Quick or Heap Sort), or the day's file is nearly ranked (an O(n + I) insertion pass would then win, as at C5, n = 1,000).

2. Shift queue (K = 152): ship the size-K heap. Evidence: 0.016 s vs 0.029 s (buffer) and 0.344 s (full sort) at n = 50,000, K = 152; cost Theta(n + K log K log(n/K)) (M5), and only O(K) memory. Assumption: K is much smaller than n and arrival order is not adversarial. Wrong if: K approaches n/10 or more (full sort catches up, measured near K = 5,000 at n = 10,000), or every record beats the current worst (reverse order: n log K work). Separately, K = 152 captures 0 frauds, so the platform lead should consider K of about 2,000 (half the frauds) if analyst time allows.

3. Live refresh: ship the size-K heap at 10-60 minute windows, strategy (c) merge into top-K below roughly 1-minute windows. Evidence: (b) 0.058 s (1 h) and 0.115 s (10 min) vs 7.1 s and 42.8 s for re-ranking; memory 65-380 KB vs 1.05 MB. At 1-minute windows (c) is 6x faster (0.17 s vs 1.02 s). Assumption: windows hold many more records than K and the heap snapshot each window is affordable. Wrong if: windows become very short (w beyond ~145 by comparisons, ~1,000-2,880 by time), or only a few windows are needed, where one re-rank is simplest.

Bonus: not attempted. **AI-use disclosure:** I used an AI assistant (Claude) for the notebook scaffolding (data preparation, benchmark harness, plotting) and to draft wording of derivations and this report. I ran the experiments in Colab, verified every contender against sorted(), and I can explain and modify each algorithm and derivation in the viva. The prediction log (saved with timestamps in prediction_log.txt) is attached as the appendix.